Processingをはじめよう

第4章

変数

目次

- 最初の変数
- 変数の作成
- 変数の処理
- 変数と計算
- 繰り返し
- Robot 2：Variables

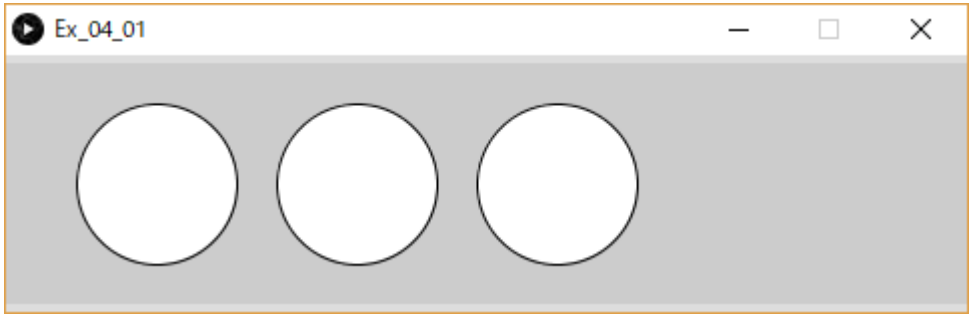
変数

- 値をメモリに保存して、プログラム中で再利用する
  - 不要な繰り返しを避ける
  - 修正が容易なコードになる
- 変数の宣言、代入、参照、スコープを理解
- Processingの特殊な変数を理解

最初の変数

- Example 4-1 同じ値の再利用

```
size(480, 120);
int y = 60;
int d = 80;
ellipse( 75, y, d, d);
ellipse(175, y, d, d);
ellipse(275, y, d, d);
```



直径の値80を変数dに代入し、再利用

y座標の値60を変数yに代入し、再利用

- 変数の宣言

```
データ型　変数名;
データ型　変数名 = 値;
```

最初の変数

- 変数名について
  - 半角英数文字, アンダースコア(_)を使う
    - ただし先頭に数字は使わない
    - 予約語を使わない
- データ型（P227）

名称	扱うデータ	値の範囲
int	整数	-2147483648 ~ 2147483647
float	浮動小数点数	-3.40282347E+38 ~ 3.40282347E+38
boolean	論理値，真偽値	true または false
char	文字（1文字）	A ~ Z, a ~ z, 0 ~ 9, 記号
String	文字列	日本語を含むあらゆる文字
color	色	color()関数を参照
PImage	画像	PNG, JPG, GIF形式の画像
PFont	フォント	-
PShape	SVGファイル	-

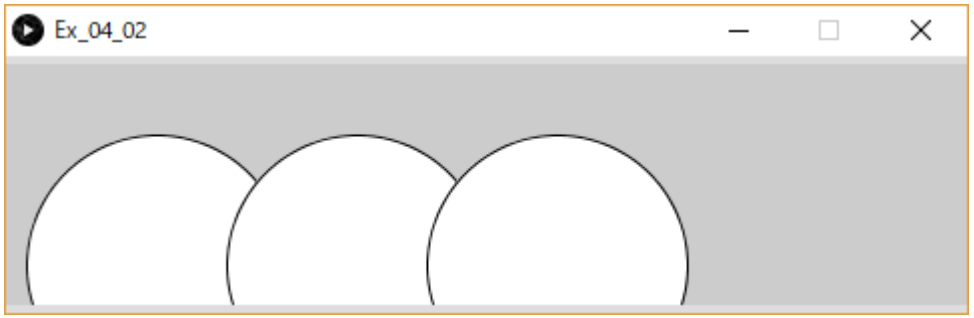
特殊な変数・変数のスコープ

- 教科書 P225
  - width, height以外の特殊な変数
  - frameRate 1秒間に画面が更新される値
  - frameCount 累積のフレーム数
  - マウスやキーボードの状態を保持する変数（5章で扱う）
- 教科書 P226
  - 変数の有効範囲（スコープ）
    - ブロック{}内で宣言した変数はブロック内で有効
    - setup()やdraw()の外で宣言した変数はどこでも有効（グローバル変数）

最初の変数

- Example 4-2 値の変更

```
size(480, 120);
int y = 100;
int d = 130;
ellipse( 75, y, d, d);
ellipse(175, y, d, d);
ellipse(275, y, d, d);
```



Example 4-1から初期値を変更

- Example 4-1 から円の位置、直径を変更
  - y を 60 から 100 に変更
  - d を 80 から 130 に変更
- 変数を使うことで、修正が容易なコードになる

変数の作成

- 整数型の変数名xを宣言 `int x;`
- 整数型の変数名xを宣言，12を代入

```
int x;
x = 12;
```

2行で

```
int x = 12;
```

1行で

- 変数の代入 `変数 = 式;`

左辺の変数に右辺の計算結果や値を保存する

変数の作成

- P42 エラーについて
  - 同じ名前で2度宣言するとエラー

```
int x;
int x = 12;
```

○ 1回目の宣言

×2回目の宣言:エラー

  - int型変数へ浮動小数点数を代入するとエラー

```
int x = 12.2;
```

× 小数点のある数を整数型に代入：エラー

```
float x = 12;
```

○ float型変数に
整数を代入できる（自動的に変換）

正しく代入するために値の変換が必要　教科書P228参照

変数の処理

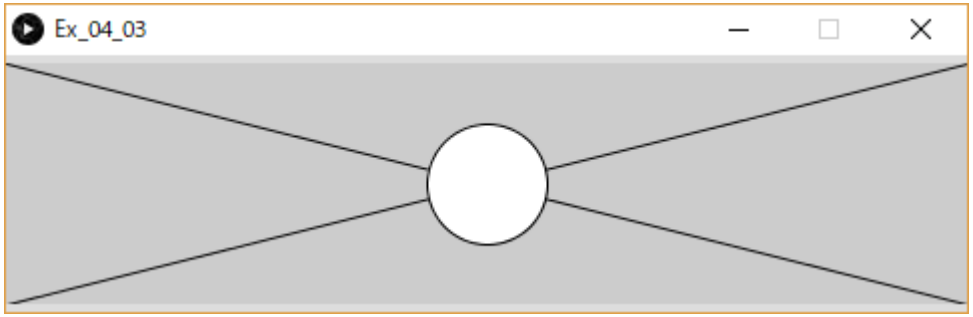
- Example 4-3 ウィンドウサイズに合わせる

```
//EX_04_03
size(480, 120);
line(0, 0, width, height);
line(width, 0, 0, height);
ellipse(width/2, height/2, 60, 60);
```

システム変数 P225
width　：ウィンドウの幅
height　：ウィンドウの高さ

- 左上(0,0)から右下(width, hegith)に直線を引く
- 右上(width, 0)から左下(0, height)に直線を引く
- 中央(width/2, height/2)に直径60の円を描く

- ウインドウサイズを変更して確認



変数と計算

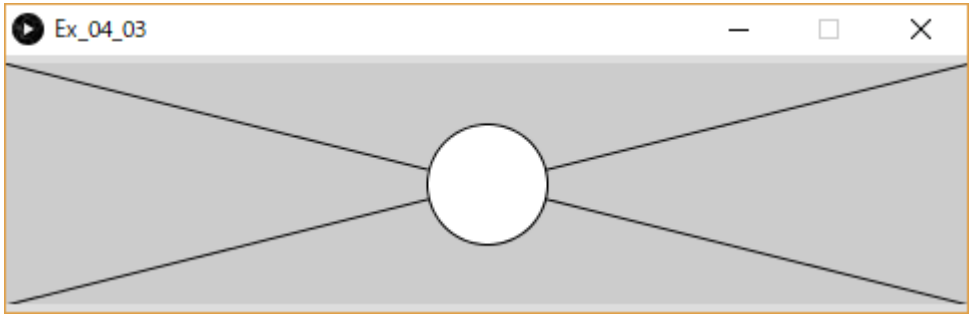
- 演算子（表の上が優先）P229

名前	記号	例
カッコ	( )	a * (b + c) カッコ内の演算が優先
単項	++ インクリメント -- デクリメント ! 論理否定	a++ aを1増やす --b bを1減らす !c cの否定
乗除	* 乗算 / 除算 % 剰余	a * b 乗算 a / b 除算 a % b 剰余（割り算の余りを計算する）
加減	+ -	a + b a - b
関係	< <= > >=	if(a > b)
等号	== 等しい != 等しくない	if(a == b) aとbが等しい場合 if(a != b) aとbが等しくない場合
論理積	&&	if(mousePressed && (a > b)) 2つの条件を同時に満たす
論理和	||	if(mousePressed || (a > b)) 2つ条件のどちらか一方を満たす
代入	= += -= *= /= %=	a = 44 aに44を代入する a += 44 aに44を加算する(a = a + 44)

変数と計算

- Example 4-4 基本的な計算

```
//EX_04_04
size(480, 120);
int x = 25;
int h = 20;
int y = 25;
rect(x, y, 300, h);
x = x + 100;
rect(x, y + h, 300, h);
x = x - 250;
rect(x, y + h*2, 300, h);
```



- 代入

```
左辺 = 右辺;
```

右辺の値や計算結果を
左辺の変数に代入する

```
x = x + 100;
```

(右辺の)xの値と100を足し
結果を(左辺の)xに代入

変数と計算

- 変数に式の計算結果を代入

```
int x = 4 + 4 * 5;
```

4+4*5の結果24をxに代入

- 関数のパラメータの欄に変数や計算式を入れることができる

```
rect(x, y+h*2, 300, h);
```

座標(x, y+h*2)に幅300,高さhの長方形を描く

- 計算と代入の短縮形

```
+= -= *= /= %= ++ --
```

繰り返し

- 繰り返し for, while P237

```
for(初期値 ; 条件式 ; 更新){
    繰り返す命令をここに書く
}
```

- 線を7本描く
  - Example 4-5 forを使わない例
  - Example 4-6 forを使った例
- 同じような処理が続く場合は、繰り返しで書けないか考えてみよう

繰り返し

- Example 4-5 何度も同じことを繰り返す

```
//EX_04_05
size(480, 120);
strokeWeight(8);
line( 20, 40,  80, 80);
line( 80, 40, 140, 80);
line(140, 40, 200, 80);
line(200, 40, 260, 80);
line(260, 40, 320, 80);
line(320, 40, 380, 80);
line(380, 40, 440, 80);
```

同じ実行結果

- Example 4-6 forループを使う

```
//EX_04_06
size(480, 120);
strokeWeight(8);
for (int i = 20; i < 400; i += 60) {
  line(i, 40, i + 60, 80);
}
```

Example 4-5

```
size(480, 120);
strokeWeight(8);
line( 20, 40,  80, 80);
line( 80, 40, 140, 80);
line(140, 40, 200, 80);
line(200, 40, 260, 80);
line(260, 40, 320, 80);
line(320, 40, 380, 80);
line(380, 40, 440, 80);
```

line()を7回続けている
ここを繰り返しを使って変更

```
size(480, 120);
strokeWeight(8);
line( 20, 40,  80, 80);
line( 80, 40, 140, 80);
line(140, 40, 200, 80);
line(200, 40, 260, 80);
line(260, 40, 320, 80);
line(320, 40, 380, 80);
line(380, 40, 440, 80);
```

赤字
共通箇所

+60
+60
+60
+60
+60
+60

+60

青字
異なる所

青字
異なる所

Example 4-6

```
size(480, 120);
strokeWeight(8);

for (int i = 20; i < 400; i += 60) {
  line( i, 40, i + 60, 80);
}
```

変数の宣言
初期値20を代入

変数iが400より小さいが真なら{}内を実行

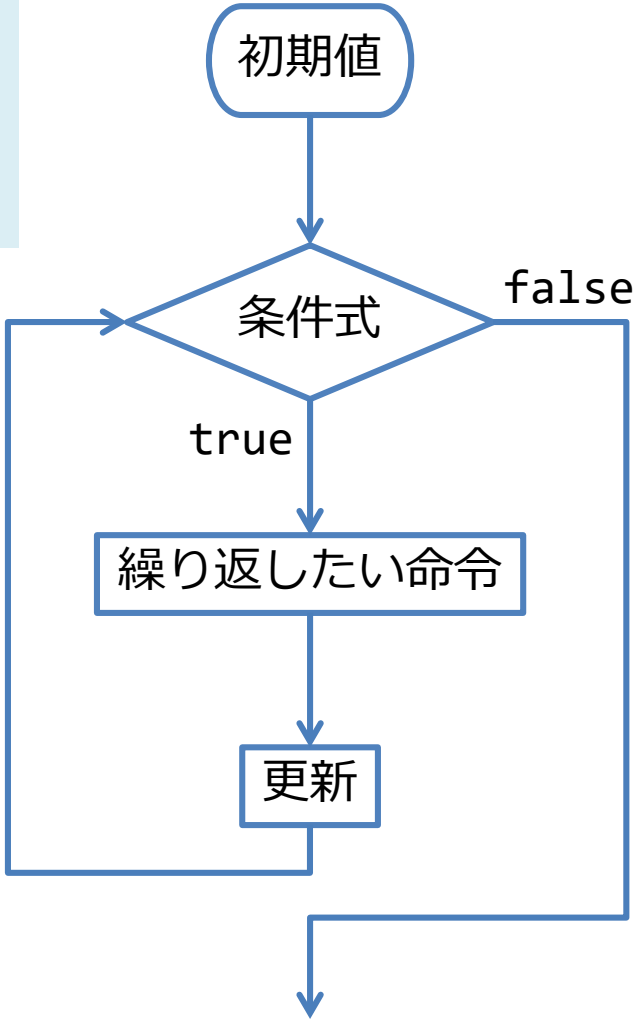
変数iを60加算して更新する

共通箇所はそのまま

異なる所は変数で表現

フローチャート

```
for(初期値 ; 条件式 ; 更新){
   繰り返したい命令;
}
```

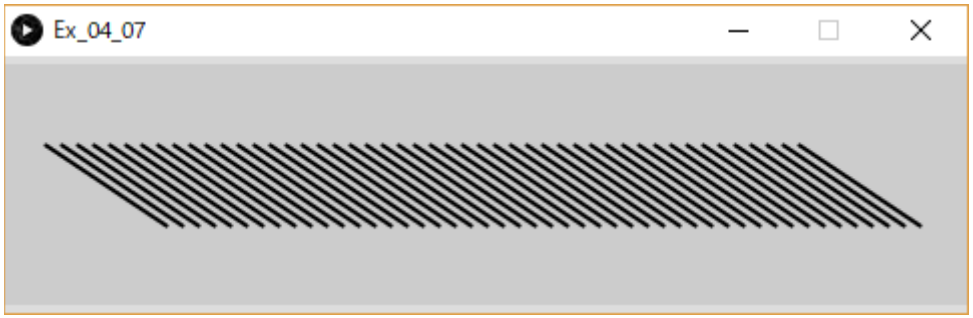


- 条件の書き方
  - 左辺と右辺を関係演算子で比較する
  - 例1）i < 400
  - 例2）x >= 200

繰り返し

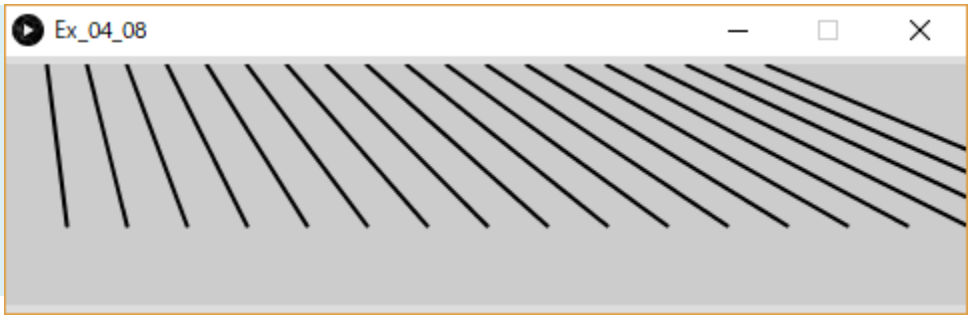
- Example 4-7 forループで柔軟体操

```
//EX_04_07
size(480, 120);
strokeWeight(2);
for (int i = 20; i < 400; i += 8) {
  line(i, 40, i + 60, 80);
}
```



- Example 4-8 扇状に広がるライン

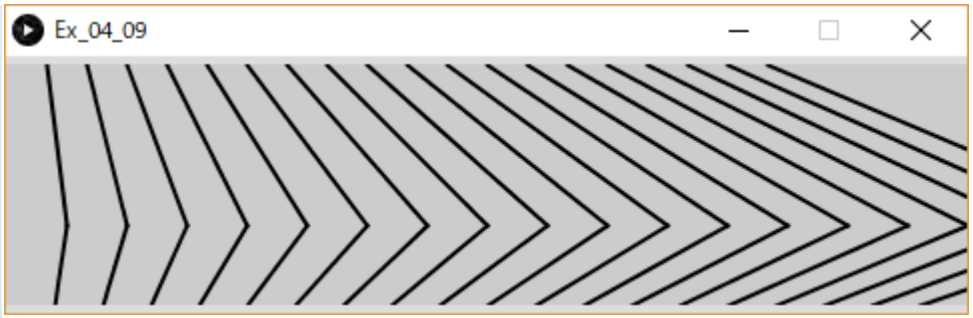
```
//EX_04_08
size(480, 120);
strokeWeight(2);
for (int i = 20; i < 400; i += 20) {
  line(i, 0, i + i/2, 80);
}
```



繰り返し

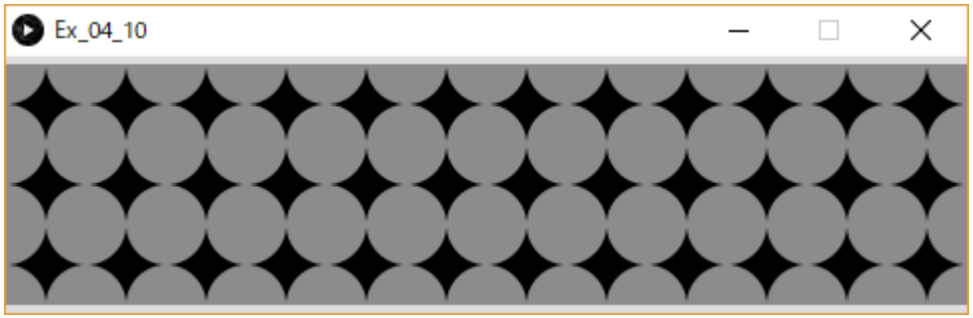
- Example 4-9 よじれるライン

```
//EX_04_9
size(480, 120);
strokeWeight(2);
for (int i = 20; i < 400; i += 20) {
  line(i, 0, i + i/2, 80);
  line(i + i/2, 80, i*1.2, 120);
}
```



- Example 4-10 ループにループを埋め込む

```
//EX_04_10
size(480, 120);
background(0);
noStroke();
for (int y = 0; y <= height; y += 40) {
  for (int x = 0; x <= width; x += 40) {
    fill(255, 140);
    ellipse(x, y, 40, 40);
  }
}
```



2重ループ

```
size(480, 120);
background(0);
noStroke();

fill(255, 140);
ellipse(0, 0, 40, 40);
fill(255, 140);
ellipse(0, 40, 40, 40);
fill(255, 140);
ellipse(0, 80, 40, 40);
fill(255, 140);
ellipse(0, 120, 40, 40);
fill(255, 140);
ellipse(0, 160, 40, 40);
(以下略)
```

fill()とellipse()を使って
縦・横に円を描き続けている

```
size(480, 120);
background(0);
noStroke();

fill(255, 140);
ellipse(0, 0, 40, 40);
fill(255, 140);
ellipse(0, 40, 40, 40);
fill(255, 140);
ellipse(0, 80, 40, 40);
fill(255, 140);
ellipse(0, 120, 40, 40);
fill(255, 140);
ellipse(0, 160, 40, 40);
(以下略)
```

+40
+40
+40
+40
+40

赤字
共通箇所

青字
異なる所

2重ループ

```
size(480, 120);
background(0);
noStroke();

for (int x = 0; x <= width; x += 40) {
   fill(255, 140);
   ellipse(x,   0, 40, 40);
}
for (int x = 0; x <= width; x += 40) {
   fill(255, 140);
   ellipse(x,  40, 40, 40);
}
for (int x = 0; x <= width; x += 40) {
   fill(255, 140);
   ellipse(x,  80, 40, 40);
}

(以下略)
```

for文を複数回
繰り返している

```
size(480, 120);
background(0);
noStroke();

for (int x = 0; x <= width; x += 40) {
   fill(255, 140);
   ellipse(x,   0, 40, 40);
}
for (int x = 0; x <= width; x += 40) {
   fill(255, 140);
   ellipse(x,  40, 40, 40);
}
for (int x = 0; x <= width; x += 40) {
   fill(255, 140);
   ellipse(x,  80, 40, 40);
}

(以下略)
```

+40

+40

+40

赤字
共通箇所

青字
異なる所

2重ループ

```
size(480, 120);
background(0);
noStroke();
```

縦方向に40間隔でheightまで繰り返し

```
for (int y = 0; y <= height; y += 40) {
```

横方向に40間隔でwidthまで繰り返し

```
  for (int x = 0; x <= width; x += 40) {
    fill(255, 140);
    ellipse(x, y, 40, 40);
  }
}
```

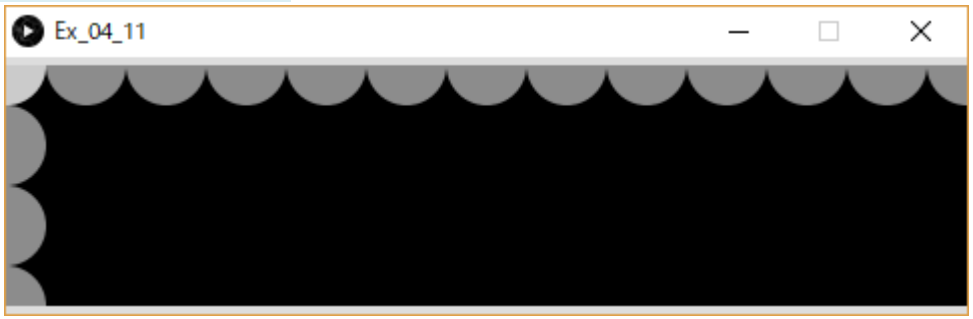
異なる所は
変数で表現

共通箇所は
そのまま

繰り返し

- Example 4-11 横と縦の列

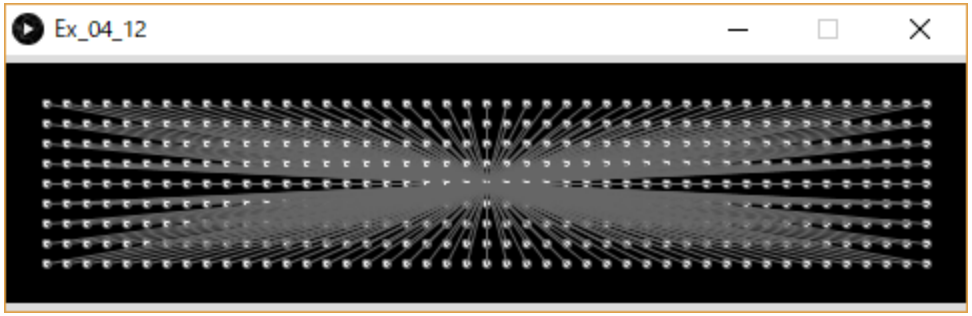
```
//EX_04_11
size(480, 120);
background(0);
noStroke();
for (int y = 0; y < height+45; y += 40) {
  fill(255, 140);
  ellipse(0, y, 40, 40);
}
for (int x = 0; x < width+45; x += 40) {
  fill(255, 140);
  ellipse(x, 0, 40, 40);
}
```



繰り返し

- Example 4-12 ピンと線

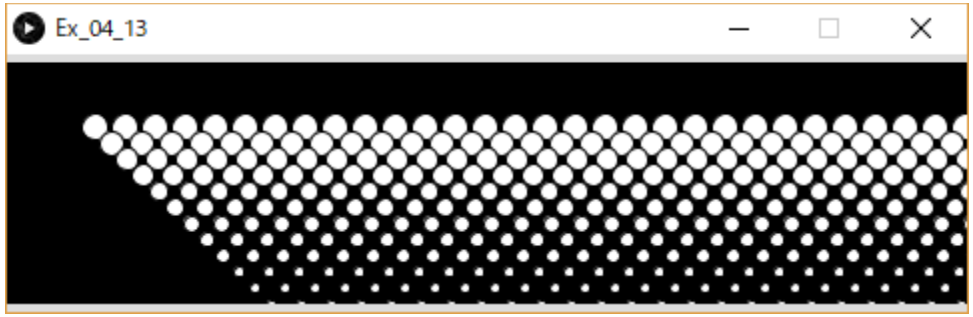
```
//EX_04_12
size(480, 120);
background(0);
fill(255);
stroke(102);
for (int y = 20; y <= height-20; y += 10) {
  for (int x = 20; x <= width-20; x += 10) {
    ellipse(x, y, 4, 4);
    line(x, y, 240, 60);
  }
}
```



繰り返し

- Example 4-13 網点

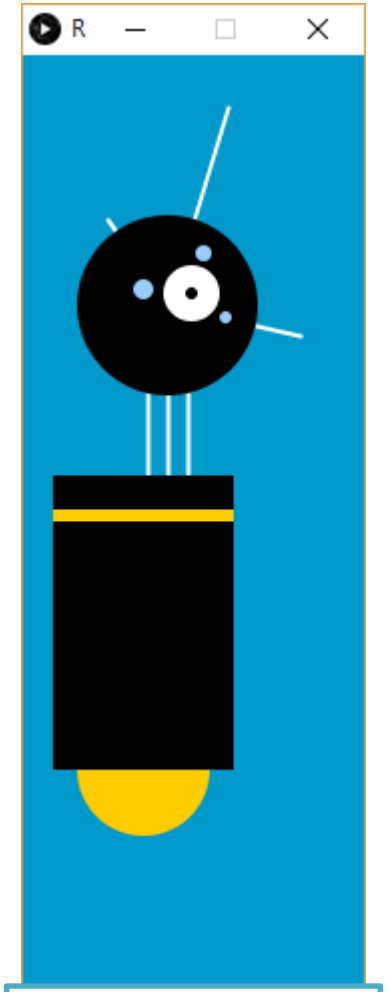
```
//EX_04_13
size(480, 120);
background(0);
for (int y = 32; y <= height; y += 8) {
  for (int x = 12; x <= width; x += 15) {
    ellipse(x + y, y, 16 - y/10.0, 16 - y/10.0);
  }
}
```



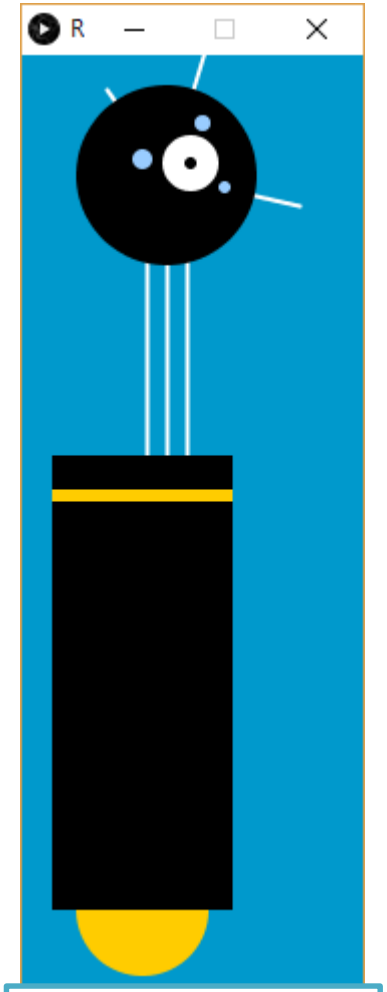
Robot 2 : Variables

- Robot1から変数を使ったプログラムに修正
- 以下の変数の値を変更するとロボットの形が変わる
  - y　　y座標
  - bodyHeight　　ロボットの胴の長さ
  - neckHeight　　ロボットの首の長さ

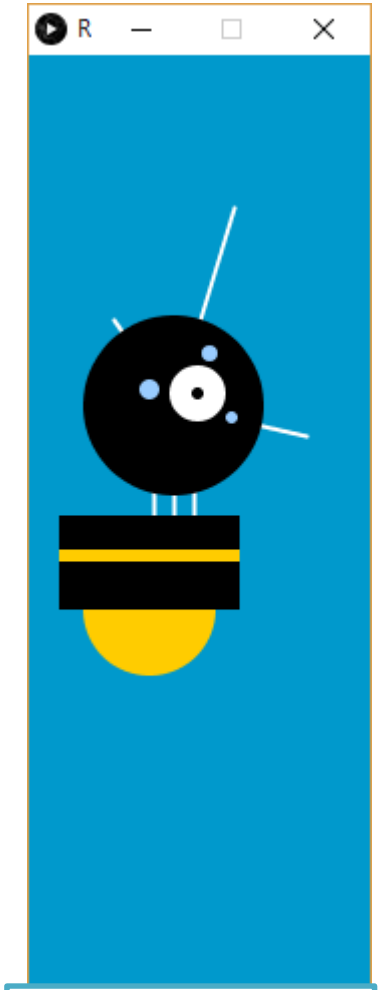
Robot 2 : Variables



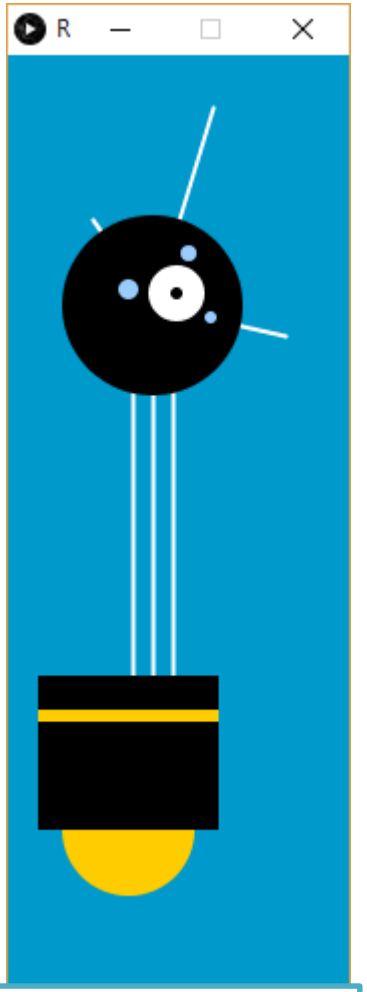
```
y = 390;
bodyHeight = 180;
neckHeight = 40;
```



```
y = 460;
bodyHeight = 260;
neckHeight = 95;
```



```
y = 310;
bodyHeight = 80;
neckHeight = 10;
```



```
y = 420;
bodyHeight = 110;
neckHeight = 140;
```